



Product Dissection & Schema Design for ZeroDha KITE

Project Summary -

The ZeroDha KITE Capstone Project aims to reverse-engineer India's largest trading platform to demonstrate proficiency in Data Architecture. The primary objective is to translate core product features such as Real-time Trading and Portfolio Tracking into a robust Relational Database Schema.

The project will focus on:

- **Product Analysis:** Identifying how ZeroDha solves industry problems like high costs and fragmented technology.
- **Schema Engineering:** Designing a normalized data structure with foundational entities (*User, Instrument, Order, Holding*).
- **Optimization Strategy:** Developing a logic-proof model that optimizes system speed by separating immutable transaction logs from real-time holdings.

The final output will be a detailed Entity-Relationship (ER) Diagram visualizing the backend logic of a high-frequency financial platform.

Problem Statement -

The objective of this Case Study was to bridge the gap between front-end product features and back-end data management. The assignment required selecting a prominent, leading platform and performing a deep-dive research into its functionality.

The primary challenge was to translate these observations into a technical schema design. This involved identifying key entities, defining specific attributes, and mapping complex relationships. The ultimate goal was to demonstrate how robust data architecture drives a platform's success and to present these findings through a detailed schema design and analysis.

Main Project Report

1. Platform Understanding (Research)

(This section covers: "Thoroughly researched and identified a top leading platform" and "Analyzed the platform's core features, functionalities, and user interactions.")

Chosen Platform: Zerodha KITE, India's leading financial technology platform for stock trading and investment.

Core Features & Functionalities: My research identified the platform's core functionalities and user interactions:

- **User Profiles:** The main account for each user.
 - **Watchlists:** Users create custom lists to monitor stocks, as seen in the app.
 - **Instruments:** The stocks themselves (like "RELIANCE" or "TCS").
 - **Orders:** The core user action of buying or selling.
 - **Portfolio (Holdings):** A summary of all stocks the user owns.
 - **Bids:** The live market prices, which are made up of user orders.
-

2. Problem-Solving Analysis

(This section covers: "Explored real-world challenges addressed by the platform's unique features.")

KITE's design is a direct solution to the significant real-world challenges that existed in the Indian brokerage industry.

- **Problem 1: High Costs for Small Investors**
 - **Challenge:** Traditional brokers charged high, percentage-based fees that created a barrier for new investors and made active trading unprofitable.
 - **KITE's Solution:** A "discount brokerage" model. By offering zero brokerage on equity delivery and a flat fee (₹20) for all other trades, KITE made investing accessible.
 - **Problem 2: Fragmented Technology**
 - **Challenge:** Traders needed multiple, slow, clunky apps, one for trading, one for checking their portfolio, and another for charting.
 - **KITE's Solution:** A single, lightweight, web-first platform that unifies real-time trading, advanced charting, and a portfolio ("Console") in one seamless interface.
 - **Problem 3: Monitoring Prices**
 - **Challenge:** Most people can't monitor stock prices continuously from 9 AM to 3 PM. If a stock dips to a desired price and they're not watching, they miss the opportunity to buy.
 - **KITE's Solution:** KITE introduced the Good Till Triggered (GTT) order. This allows a user to set a trigger (like "Buy Reliance if it drops to 2000") and this order stays active for **one full year**. It works like a robot watching the market for you.
-

3. Case Study Development

(This section covers: "Developed a comprehensive case study showcasing how the platform solves specific user difficulties.")

KITE's features are designed to solve specific user difficulties:

- **User Difficulty:** "The stock market has thousands of stocks, it's overwhelming."
 - Platform Solution: The Watchlist feature. This is the core solution to information overload. It lets a user ignore the noise and build their own simple, custom view of the 10-20 stocks they personally care about.
 - **User Difficulty:** "I want to buy a stock, but I don't want to watch the screen all day."
 - Platform Solution: The GTT (Good Till Triggered) Order. A user can set a long-term "trigger" (e.g., "Buy 100 shares of Reliance if the price hits ₹2500") and the order stays active for one year. This "set it and forget it" feature solves the daily monitoring problem completely.
 - **User Difficulty:** "How do I know what I own and if I'm making money?"
 - Platform Solution: The Portfolio screen. This provides a simple, one-glance summary of all "Holdings," their current value, and the total profit or loss. It solves the problem of users having to manually track their investments in a spreadsheet.
 - **User Difficulty:** "I want to run a complex strategy with multiple orders."
 - Platform Solution: The Baskets feature. This allows a user to group multiple orders (e.g, a 4-leg options strategy) into a single "basket" and execute it with one click, solving the problem of manual, one-by-one execution.
-

4. Schema Design & Expanded Description

(This section covers: "Crafted a schema design that aligns with the chosen platform's data structure." This detailed breakdown provides the necessary depth for the video presentation.)

Based on the analysis, I designed a schema with the following key entities (tables) and their attributes.

User Entity

- *(Stores the customer's main profile.)*
- **UserID** (Primary Key): A unique ID for every user, like "AB1234".
- **Username**: The user's login ID.
- **Email**: Used for communication and password resets.
- **Full_Name**: The user's real name, for legal and bank records.
- **Demat_ID**: The 16-digit ID that legally links them to their stocks.

Instrument Entity

- *(This is the "master list" of all tradable stocks, like **ASTRON** or **ASIANPAINT**.)*
- **InstrumentID** (Primary Key): A unique ID for every stock.
- **Trading_Symbol**: The ticker symbol, like "RELIANCE" or "TCS".
- **Exchange**: Tells the system if this is an "NSE" or "BSE" stock.
- **Full_Name**: The full company name, like "Reliance Industries Ltd."

Watchlist Entity

- *(This table stores the names of the user's watchlists, like "Favorites" or "My list".)*
- **WatchlistID** (Primary Key): A unique ID for the watchlist.
- **UserID** (Foreign Key): Links the watchlist to a specific user.
- **Watchlist_Name**: The name the user gives it.

Watchlist_Item Entity

- *(This is a junction table that connects watchlists and stocks.)*
- **Watchlist_ItemID** (Primary Key): A unique ID for the row.

- **WatchlistID** (Foreign Key): Links to the **Watchlist** table.
- **InstrumentID** (Foreign Key): Links to the **Instrument** table.
- *(Rationale: This table manages the many-to-many relationship. A user can have many stocks on one watchlist, and one stock can be on many watchlists.)*

Order Entity

- *(This is the most critical table, acting as a logbook of all actions.)*
- **OrderID** (Primary Key): A unique ID for every single click of the "buy" or "sell" button.
- **UserID** (Foreign Key): Links to the user who placed the order.
- **InstrumentID** (Foreign Key): Links to the stock they want to trade.
- **Transaction_Type**: A simple field, "BUY" or "SELL".
- **Product_Type**: A key business rule. "CNC" (Cash n Carry) means this is a long-term investment. "MIS" (Margin Intraday) means this is a short-term trade. This one column decides if the stock goes to the Portfolio.
- **Quantity**: How many shares.
- **Price**: The price the user wants (for a "Limit" order).
- **Trigger_Price**: The special price that activates a "GTT" or "Stoploss" order.
- **Status**: The live status, like "OPEN", "EXECUTED", or "CANCELLED".

Holding Entity

- *(This is the user's final Portfolio, summarizing what they own.)*
 - **HoldingID** (Primary Key): A unique ID for the holding record.
 - **UserID** (Foreign Key): Links to the user.
 - **InstrumentID** (Foreign Key): Links to the stock they own.
 - **Quantity**: The *total* number of shares they own.
 - **Average_Buy_Price**: The weighted average price they paid for all those shares. This is critical for calculating their total Profit or Loss.
-

5. Rationale and Strategy

(This section covers: "Provided a clear rationale for the schema design choices made," including relationships and business logic.)

This schema is designed for speed, accuracy, and scalability. The rationale is broken down into two parts: the relationships and the core business logic.

(A) Relationships & Integrity Constraints

This defines the *rules* of how the tables are connected.

- User → Orders (One-to-Many): One **User** can place *many* **Orders**. This links all actions to a customer.
- User → Holdings (One-to-Many): One **User** can have *many* **Holdings** (stocks) in their portfolio.
- Instrument → Orders (One-to-Many): One **Instrument** (like "ASTRON") can be part of *many* different **Orders** from many different users.
- Watchlist ↔ Instrument (Many-to-Many): One **Watchlist** can contain *many* **Instruments**, and one **Instrument** can be on *many* **Watchlists**. This is why we *must* use the **Watchlist_Item** junction table to link them.

(B) Core Business Logic Rationale

This explains the *business purpose* of the key tables.

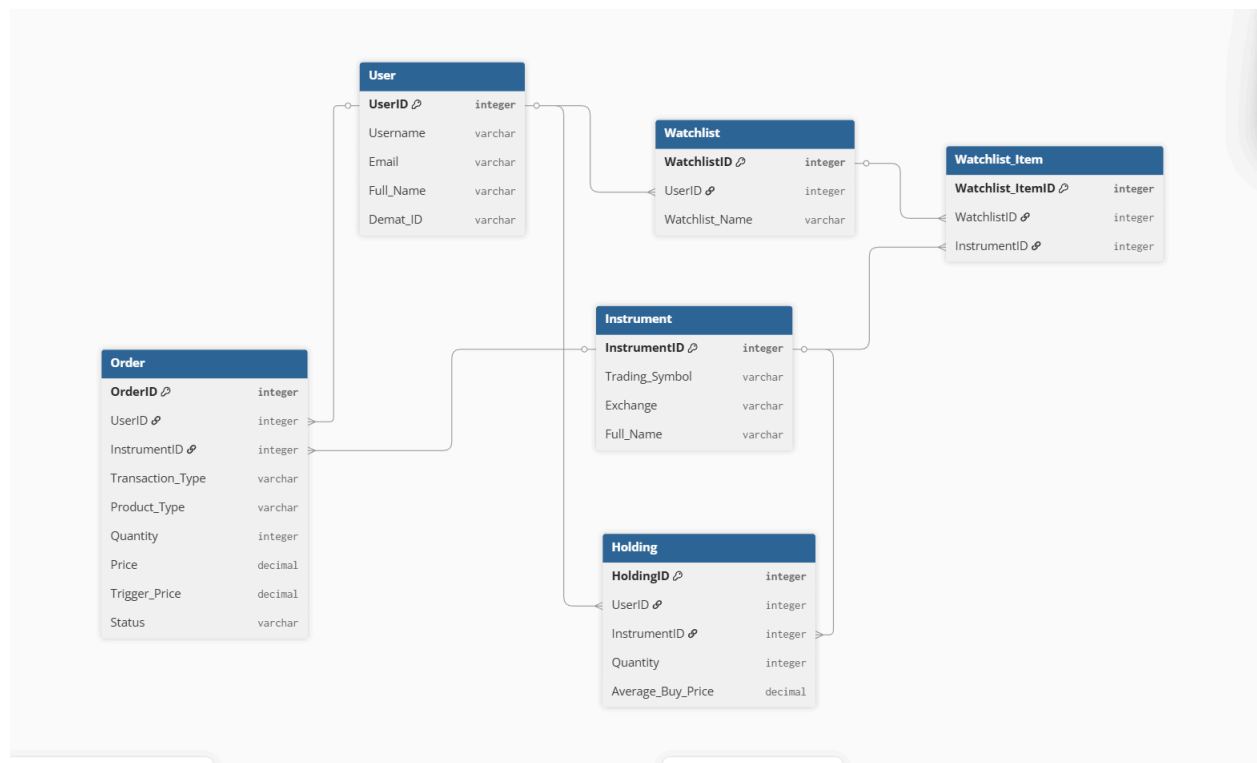
- **Instrument** as a "Single Source of Truth": Instead of typing "Reliance Industries Ltd." in thousands of **Order** records, the system just uses the **InstrumentID**. If the company's name changes, you only update it *one time* in the **Instrument** table. This prevents errors and saves massive amounts of space.
- The **Order** vs. **Holding** Separation (The Core Logic): This is the most important design choice.
 - The **Order** table is the "Transaction Log." It is an *immutable ledger*. It's a complete history of every *attempt* to trade. You never delete from this table; you only add. This is essential for financial auditing and for the user's "Tradebook."

- The **Holding** table is the "Current State." It is a *summary*. It only shows what the user *owns right now*. When a "BUY" **Order** with **Product_Type** = "CNC" is executed, a process *updates* the **Holding** table. When a "SELL" **Order** is executed, the **Holding** table is updated again.
 - Why separate them? Speed. A user checks their portfolio 50 times a day. You cannot read their entire 10-year order history 50 times a day. The app just reads the small, simple **Holding** table, making it instant. This separation of a *log* (**Order**) from a state (**Holding**) is a foundational concept in database design for high-speed applications.
-

6. ER Diagram (Visual Representation)

(This section covers: "Created an accurate and illustrative Entity-Relationship (ER) diagram.")

This ER diagram visualizes the schema design, showing how the **User** is the central entity and connects to all their associated data.



conclusion

In this case study, we delved into the design of Zerodha KITE's schema and its Entity-Relationship diagram. Zerodha has revolutionized the Indian financial landscape by solving key user problems like high costs and fragmented technology.

The platform's intricate data model, consisting of entities like **User**, **Instrument**, **Order**, **Holding**, and **Watchlist**, forms the foundation for its seamless and high-speed functionality.

By understanding this schema, we gain insight into how KITE effectively manages the complexities of real-time financial transactions, user portfolio management, and data integrity, contributing to its widespread popularity and continued growth in the world of financial technology.